

GraphQL: Democratising API Queries, the New Normal

This article introduces you to GraphQL, with a step-by-step example in a Windows 10 environment, so that you are well equipped with it to navigate this 'new normal' yourself.



As we are steadily going through the phases of unlock, you may consider visiting a restaurant with friends or family after a long time. In the restaurant, you will typically get two choices: buffet or *à la carte*. In a buffet, you go to various tables, collect the food you choose and come back to your table. In *à la carte*, on the other hand, you order as per your wishes and then eat. The waiter takes multiple orders from your table and delivers on a single visit. Sounds familiar?

This is exactly what GraphQL is! It is a query *à la carte* that aggregates related and non-related resources from various database tables in servers to deliver a single, conglomerated query to the client. Of course, the client can decide how many requests to make, based on various factors, to get the information required. Hence, GraphQL promises a paradigm shift to empower the client to decide on how

data is delivered from the server. That is unlikely in REST, where the server dictates how many REST APIs are to be called at what endpoints to retrieve the required information. So, the real power of GraphQL is to decentralise and democratise the request query to make it the client's choice with greater flexibility.

The story so far

We have already introduced GraphQL. Now let us quickly look at its journey, primarily in the context of Facebook.

GraphQL was started as a sideways project, SuperGraph, by Facebook way back in 2012. It kept on evolving around mid-2015, when it was open sourced with the 'Technical Preview' mode. This led to its 'Production Ready' mode in September 2016, almost four years ago. Figure 1, courtesy of GraphQL/Facebook's Lee Byron's talk on YouTube, depicts the evolution timeline vividly.

A helicopter view

We have already talked about the empowerment and decentralisation of the request query mechanism as the fulcrum of the GraphQL's value proposition. Let's understand Figure 2 to get a better overview of GraphQL before delving into the details.

As shown in the figure, the client is making a single http network call, which spans across different database tables in the server, and is typically exposed as four different endpoints through REST API. In GraphQL, the client is making a single POST API call (`POST <server_ip>/graphql -d {query: {...}}`) to aggregate the data across four different server tables to return a single response to the client. Now, it is not necessary that the request must be one. Rather, it is up to the client to decide how many requests are to be made, which may be based on the scale or some other parameters. So, the client gets whatever is required, instead of getting it piecemeal as in REST, along with a type safety inbuilt into GraphQL. The latter is generic, not tied to a particular client as such, and widely used in the industry (e.g., GitHub, Facebook, and many more).

Getting hands dirty in Windows 10

Let us look at the little 'hello world' GraphQL script as a baby step. We are working with Windows 10. However, the process is equally possible with Linux and Mac also.

